Esta es una copia de una conversación compartida de ChatGPT
Denunciar conversación

ayer 10:09 a.m.

Se cargó un archivo

I am implementing a technical assessment using a Spec-Driven Development approach.

The attached Project Vision document is the approved project scope.

Do not generate any code.

Review the document, identify ambiguities, inconsistencies, or missing functional decisions.

Ver más

Se cargó un archivo

Context

We are developing this solution following a Spec-Driven Development (SDD) approach.

The attached Project Vision (version 0.2) has already been reviewed and approved after an architectural review. Consider it the single source of truth for the project vision.

Your task is NOT to generate code.

Ver más

ayer 1:31 p.m.

Se cargó un archivo

We continued the work in a separate working conversation and completed **05_Architecture.md Version 1.0**.

Please treat the following document as the completed architectural specification derived from the previously approved baseline:

* 01_ProjectVision.md v0.2 (Approved)
* 02_FunctionalSpecification.md v1.2 (Approved)
* 03_DomainModel.md v1.0 (Approved)
* 04_APIContract.md v1.0 (Approved)

Ver más

Thank you. Those editorial refinements are well aligned with the intent of this document, and I will incorporate them into the final assembled version.

Specifically:

- Throughout the document, I will prefer descriptive language such as "**adopts**", "**uses**", "**implements**", or "**is organized**" over prescriptive language such as "**shall**" wherever appropriate. This better reflects that ADRs document accepted architectural decisions rather than establish new requirements.
- In **ADR-005**, I will explicitly clarify that the combination of **Entity Framework for reads** and **Dapper for writes** is an architectural decision made for **this project**, based on its approved Technical Assessment Specification and Architecture, and is **not intended as a universal recommendation** for all systems.

No architectural content will change.

06_ArchitectureDecisionRecords.md

Version: 1.0
Status: Draft (Pending Review)

4. Architecture Decision Records (continued)

ADR-008 — Use SQL Server as the Relational Database

Status

Accepted

Context

The approved Technical Assessment Specification identifies Microsoft SQL Server as the relational database platform for the Inventory Management System.

The selected persistence platform must support transactional consistency, relational integrity, mature tooling, and seamless integration with the technologies adopted by the approved Architecture.

The database technology is an implementation decision and does not affect the externally observable behavior defined by the approved specifications.

Decision

The solution **uses Microsoft SQL Server** as its relational database management system.

SQL Server stores the persistent data required by the approved Domain Model and supports the persistence strategy defined by the approved Architecture.

The database platform is selected as part of the project's implementation approach and remains transparent to API consumers.

Alternatives Considered

PostgreSQL

Provides a mature, standards-compliant relational database platform.

Rejected because the Technical Assessment Specification selected SQL Server as the implementation platform for this project.

MySQL

Widely adopted and suitable for many transactional systems.

Rejected because selecting a different database platform would introduce unnecessary divergence from the approved technical baseline.

NoSQL Database

Provides flexible schema evolution and horizontal scalability.

Rejected because the approved Domain Model is strongly relational and requires transactional consistency between aggregates.

Consequences

Positive Consequences

- Mature relational database platform.
- Strong transactional support.
- Robust tooling and administration.
- Tight integration with the selected persistence technologies.
- Broad industry adoption and long-term support.

Negative Consequences

- Vendor-specific platform.
- Future migration to another relational database would require additional implementation effort.

Trade-offs

Adopting SQL Server aligns the implementation with the approved technical baseline while providing a mature and stable relational platform.

Specification Impact

This ADR records an accepted implementation decision.

It introduces no modifications to the approved:

- Project Vision
- Functional Specification
- Domain Model
- API Contract

Related Documents

- Technical Assessment Specification
- 05_Architecture.md (Persistence Strategy)

ADR-009 — Adopt JWT Bearer Authentication

Status

Accepted

Context

The approved API Contract specifies JWT Bearer authentication for protecting secured endpoints.

The implementation requires an authentication mechanism that validates bearer tokens while remaining transparent to clients beyond the behavior already defined by the API Contract.

The architectural decision concerns the implementation of authentication, not the definition of externally observable authentication behavior.

Decision

The solution **adopts JWT Bearer authentication** for securing protected API endpoints.

Authentication is implemented using bearer tokens as defined by the approved API Contract.

The implementation validates JWT tokens before protected operations are executed while preserving the authentication behavior already specified by the approved API Contract.

Alternatives Considered

Session-Based Authentication

Maintains authenticated user state on the server.

Rejected because the approved API Contract defines stateless bearer token authentication.

Cookie-Based Authentication

Provides browser-oriented authentication using cookies.

Rejected because the solution exposes a REST API intended to operate through bearer tokens.

API Key Authentication

Provides simple request authentication.

Rejected because it does not provide the identity and claims model established by the approved API Contract.

Consequences

Positive Consequences

- Stateless authentication.
- Well-established industry practice.
- Suitable for REST APIs.
- Supports secure authorization workflows.
- Consistent with the approved API Contract.

Negative Consequences

- Token lifecycle management requires appropriate implementation.
- Authentication depends on correct token validation.

Trade-offs

JWT Bearer authentication provides a balance between security, interoperability, and implementation simplicity while remaining consistent with the approved specifications.

Specification Impact

This ADR records the architectural implementation of the authentication approach already established by the approved API Contract.

It introduces no new authentication behavior or security requirements.

Related Documents

- 04_APIContract.md
- 05_Architecture.md (Security Architecture)
- Technical Assessment Specification

ADR-010 — Use FluentValidation for Request Validation

Status

Accepted

Context

The approved Architecture separates application validation responsibilities from controllers and domain entities.

A validation mechanism is required to verify application requests before business logic is executed while maintaining a clean separation of concerns.

Decision

The solution **uses FluentValidation** to validate application requests within the Application Layer.

Validation rules are implemented independently of controllers and are executed before the corresponding application handler processes the request.

This implementation supports the validation strategy described by the approved Architecture without altering the business validation rules defined by the approved specifications.

Alternatives Considered

Controller-Based Validation

Validation logic implemented directly within controllers.

Rejected because it couples validation to the Presentation Layer and reduces reuse.

Manual Validation Inside Handlers

Validation performed explicitly within each handler.

Rejected because it duplicates validation logic and reduces consistency across requests.

Custom Validation Framework

A proprietary validation mechanism.

Rejected because FluentValidation provides mature functionality that satisfies the project's implementation requirements.

Consequences

Positive Consequences

- Separation of validation concerns.

- Consistent request validation.
- Improved maintainability.
- Reduced duplication.
- Better testability.

Negative Consequences

- Additional framework dependency.
- Developers require familiarity with FluentValidation.

Trade-offs

The additional dependency is accepted because it simplifies validation while preserving a clean application architecture.

Specification Impact

This ADR records an accepted implementation decision.

Business validation rules continue to be defined exclusively by the approved specifications.

Related Documents

- Technical Assessment Specification
- 05_Architecture.md (Validation Strategy)

ADR-011 — Adopt Dependency Injection

Status

Accepted

Context

The approved Architecture emphasizes loose coupling between architectural layers and components.

A mechanism is required to manage object creation and dependency resolution while preserving the dependency direction established by Clean Architecture.

Decision

The solution **adopts Dependency Injection (DI)** as the mechanism for resolving application dependencies.

Dependencies are provided through constructor injection, allowing components to depend on abstractions rather than concrete implementations.

The dependency injection container is configured within the application's composition root, keeping dependency management separate from business logic.

Alternatives Considered

Manual Object Construction

Objects are instantiated directly throughout the application.

Rejected because it tightly couples components and complicates testing.

Service Locator Pattern

Components resolve dependencies dynamically.

Rejected because it hides dependencies and reduces code transparency.

Static Singleton Implementations

Shared services are exposed through static instances.

Rejected because they increase coupling and make testing more difficult.

Consequences

Positive Consequences

- Loose coupling.
- Improved testability.
- Clear dependency management.
- Better maintainability.
- Alignment with the Dependency Inversion Principle.

Negative Consequences

- Initial configuration effort.
- Developers must understand dependency injection concepts.

Trade-offs

The additional configuration required by Dependency Injection is accepted because it significantly improves maintainability and supports the architectural goals established for the project.

Specification Impact

This ADR records an accepted implementation decision.

It does not introduce new functional behavior or modify any approved specification.

Related Documents

- Technical Assessment Specification
- 05_Architecture.md (Dependency Injection)

End of Part 3

The final installment will include:

- **ADR-012 – Use Docker for Containerization**
- **ADR-013 – Use Docker Compose for Local Orchestration**
- **ADR-014 – Use Swagger / OpenAPI for API Documentation**
- **ADR-015 – Enforce Layer Dependency Rules Through Architecture Tests**
- **Decision Traceability Matrix**

- ADR Governance
- Final document consistency review

Part 3 is approved.

The document continues to remain fully consistent with the approved SDD baseline and maintains a clear distinction between architectural decisions and approved specifications.

No architectural inconsistencies were identified.

Please proceed with the final installment, including:

Ver más

Thank you. Based on the reviews completed so far, the document remains internally consistent with:

- 01_ProjectVision.md v0.2 (Approved)
- 02_FunctionalSpecification.md v1.2 (Approved)
- 03_DomainModel.md v1.0 (Approved)
- 04_APIContract.md v1.0 (Approved)
- 05_Architecture.md v1.0 (Approved)
- Technical Assessment Specification

The following concludes **06_ArchitectureDecisionRecords.md Version 1.0**.

4. Architecture Decision Records (continued)

ADR-012 — Use Docker for Containerization

Status

Accepted

Context

The approved Architecture identifies containerization as the preferred mechanism for providing a consistent execution environment throughout development and deployment.

A standardized packaging approach reduces environmental differences and improves deployment reproducibility without affecting the behavior defined by the approved specifications.

Decision

The solution **uses Docker** to package the application and its runtime dependencies into portable containers.

Containerization provides a consistent execution environment across supported deployment targets while remaining transparent to API consumers.

The use of Docker is an implementation decision specific to this project and is independent of the system's functional behavior.

Alternatives Considered

Native Host Deployment

Applications execute directly on the operating system.

Rejected because environmental differences can introduce inconsistencies across development, testing, and deployment environments.

Virtual Machines

Applications execute inside full virtual machines.

Rejected because virtual machines introduce greater resource overhead than required for the current project.

Alternative Container Technologies

Other container platforms provide similar capabilities.

Rejected because Docker was selected by the approved Technical Assessment Specification as the project's containerization technology.

Consequences

Positive Consequences

- Consistent runtime environment.
- Simplified deployment.
- Improved portability.
- Reproducible application packaging.
- Alignment with modern deployment practices.

Negative Consequences

- Additional tooling required for development.
- Developers require familiarity with container concepts.

Trade-offs

The additional tooling is accepted because containerization significantly improves environment consistency and deployment reliability.

Specification Impact

This ADR records an accepted implementation decision.

It introduces no changes to the approved specifications or externally observable behavior.

Related Documents

- Technical Assessment Specification
- 05_Architecture.md (Containerization Strategy)

ADR-013 — Use Docker Compose for Local Orchestration

Status

Accepted

Context

The solution consists of multiple components that must execute together during local development.

Coordinating these components manually increases configuration effort and reduces consistency between development environments.

Decision

The solution **uses Docker Compose** to orchestrate the application's local development environment.

Docker Compose coordinates the execution of the application and its supporting services through a declarative configuration.

This orchestration approach is intended solely for development and local integration scenarios and does not prescribe a production deployment strategy.

Alternatives Considered

Manual Service Startup

Each component is started independently.

Rejected because manual coordination is repetitive and prone to configuration inconsistencies.

Custom Startup Scripts

Local environments are initialized through operating system scripts.

Rejected because scripts are typically platform-dependent and more difficult to maintain.

Container Orchestration Platforms

Platforms such as Kubernetes provide advanced orchestration capabilities.

Rejected because the current project scope does not require production-grade orchestration for local development.

Consequences

Positive Consequences

- Simplified local environment setup.
- Consistent developer experience.
- Reduced manual configuration.
- Improved reproducibility.

Negative Consequences

- Additional Docker Compose configuration.
- Developers require familiarity with Docker Compose.

Trade-offs

Docker Compose provides sufficient orchestration capabilities for the project's current scope while maintaining implementation simplicity.

Specification Impact

This ADR records an accepted implementation decision.

It does not modify the approved specifications or define deployment requirements beyond the approved Architecture.

Related Documents

- Technical Assessment Specification

- 05_Architecture.md (Containerization Strategy)

ADR-014 — Use Swagger / OpenAPI for API Documentation

Status

Accepted

Context

The approved Architecture identifies API documentation as an important capability for development, testing, and integration.

A standardized documentation mechanism improves discoverability of API endpoints while remaining derived entirely from the approved API Contract.

Decision

The solution **uses Swagger/OpenAPI** to generate interactive API documentation.

The generated documentation reflects the implementation of the approved API Contract and serves as a development and integration aid.

Swagger documentation is derived from the implemented API and does not constitute the authoritative specification of system behavior.

Alternatives Considered

Manually Maintained Documentation

API documentation maintained independently.

Rejected because manually maintained documentation is more susceptible to becoming inconsistent with the implementation.

No Interactive Documentation

API consumers rely exclusively on external documentation.

Rejected because interactive documentation improves developer productivity and simplifies API exploration.

Alternative Documentation Frameworks

Other documentation generators provide similar functionality.

Rejected because Swagger/OpenAPI is the technology identified in the approved technical baseline.

Consequences

Positive Consequences

- Interactive API exploration.
- Improved developer experience.
- Reduced documentation maintenance effort.
- Better support for testing and integration.

Negative Consequences

- Additional package dependency.
- Documentation quality depends upon implementation accuracy.

Trade-offs

Swagger/OpenAPI improves API usability while remaining consistent with the approved API Contract.

Specification Impact

This ADR records an accepted implementation decision.

The approved **04_APIContract.md** remains the authoritative specification of the public API.

Swagger documentation represents the implemented API and must remain consistent with the approved API Contract.

Related Documents

- 04_APIContract.md
- Technical Assessment Specification
- 05_Architecture.md (API Documentation)

ADR-015 — Enforce Layer Dependency Rules Through Architecture Tests

Status

Accepted

Context

The approved Architecture establishes explicit dependency rules between architectural layers.

As the solution evolves, there is a risk that implementation changes could unintentionally violate these dependency rules.

An automated verification mechanism improves architectural consistency throughout the project's lifecycle.

Decision

The solution **uses architecture tests** to verify compliance with the dependency rules defined by the approved Architecture.

These tests validate architectural constraints such as permitted project references and dependency direction.

Architecture tests verify implementation structure only and do not replace functional or integration testing.

Alternatives Considered

Manual Code Reviews

Dependency rules are verified during peer review.

Rejected because manual verification is subject to human error and may not consistently identify architectural violations.

Documentation-Only Governance

Developers rely solely on architectural documentation.

Rejected because documentation alone cannot automatically detect implementation drift.

No Automated Architectural Verification

Architectural compliance depends exclusively on developer discipline.

Rejected because long-term architectural consistency benefits from automated verification.

Consequences

Positive Consequences

- Continuous architectural validation.
- Early detection of dependency violations.
- Improved maintainability.
- Better preservation of the approved Architecture.

Negative Consequences

- Additional test maintenance.
- Increased build execution time.

Trade-offs

The modest increase in testing effort is accepted because it helps preserve architectural integrity throughout the evolution of the solution.

Specification Impact

This ADR records an accepted implementation decision.

It introduces no functional behavior and does not modify any approved specification.

Related Documents

- Technical Assessment Specification
- 05_Architecture.md (Testing Strategy, Architectural Constraints)

5. Decision Traceability Matrix

ADR	Architectural Decision	Primary Reference
ADR-001	Adopt Clean Architecture	05_Architecture.md – Architectural Principles
ADR-002	Organize the Solution into Independent Projects	05_Architecture.md – Solution Structure
ADR-003	Adopt CQRS	05_Architecture.md – Application Layer Design
ADR-004	Use MediatR for Request Dispatching	05_Architecture.md – Request Processing Flow
ADR-005	Separate Read and Write Persistence Technologies	05_Architecture.md – Persistence Strategy
ADR-006	Use Entity Framework for Read Operations	05_Architecture.md – Persistence Strategy
ADR-007	Use Dapper for Write Operations	05_Architecture.md – Persistence Strategy

ADR	Architectural Decision	Primary Reference
ADR-008	Use SQL Server as the Relational Database	Technical Assessment Specification / 05_Architecture.md
ADR-009	Adopt JWT Bearer Authentication	04_APIContract.md / 05_Architecture.md
ADR-010	Use FluentValidation for Request Validation	05_Architecture.md – Validation Strategy
ADR-011	Adopt Dependency Injection	05_Architecture.md – Dependency Injection
ADR-012	Use Docker for Containerization	05_Architecture.md – Containerization Strategy
ADR-013	Use Docker Compose for Local Orchestration	05_Architecture.md – Containerization Strategy
ADR-014	Use Swagger / OpenAPI for API Documentation	04_APIContract.md / 05_Architecture.md
ADR-015	Enforce Layer Dependency Rules Through Architecture Tests	05_Architecture.md – Testing Strategy

6. ADR Governance

This document records the accepted architectural decisions that support the implementation of the approved specification baseline.

The following governance principles apply:

1. Architecture Decision Records document **accepted architectural decisions** and their rationale.
2. Architecture Decision Records do not introduce functional requirements, business rules, domain concepts, API resources, or externally observable behavior.
3. The approved specification baseline remains authoritative:
   - 01_ProjectVision.md

- 02_FunctionalSpecification.md
- 03_DomainModel.md
- 04_APIContract.md

4. The approved Architecture describes how the specification baseline is realized.
5. This document records why architectural approaches were selected; it does not redefine or extend the approved Architecture.
6. Future architectural decisions should be documented through additional Architecture Decision Records rather than by modifying existing approved ADRs, unless an approved governance process explicitly requires revision.
7. Any future architectural decision that changes externally observable behavior must first be reflected in the appropriate specification artifact before being documented as an architectural decision.
8. Architectural decisions should maintain complete traceability to the approved specification baseline and the approved Architecture.

End of Part 4 — End of Document

From my review, the assembled **06_ArchitectureDecisionRecords.md Version 1.0** is internally consistent and aligned with the established Specification-Driven Development governance. It preserves the separation between **what** the system must do (specifications), **how** it is implemented (architecture), and **why** those implementation approaches were chosen (ADRs), without introducing new functional behavior or altering the approved baseline.

The final review is complete.

I confirm that **06_ArchitectureDecisionRecords.md Version 1.0** is internally consistent and fully aligned with the approved SDD baseline, including:

* 01_ProjectVision.md v0.2
* 02_FunctionalSpecification.md v1.2
* 03_DomainModel.md v1.0
* 04_APIContract.md v1.0
* 05_Architecture.md v1.0

Ver más

Acknowledged.

I formally recognize the completion of the Specification-Driven Development baseline and acknowledge the following documents as the project's approved and authoritative specification set:

Document	Version	Status
01_ProjectVision.md	0.2	**Approved**
02_FunctionalSpecification.md	1.2	**Approved**
03_DomainModel.md	1.0	**Approved**
04_APIContract.md	1.0	**Approved**
05_Architecture.md	1.0	**Approved**
06_ArchitectureDecisionRecords.md	1.0	**Approved**

Established Governance Baseline

The project now has a complete, internally consistent Specification-Driven Development (SDD) baseline.

The roles of each artifact are clearly defined:

- **Project Vision** establishes the business objectives and scope.
- **Functional Specification** defines the required system behavior.
- **Domain Model** defines the business concepts and relationships.
- **API Contract** specifies the externally observable interface.
- **Architecture** defines how the approved behavior is realized.
- **Architecture Decision Records (ADRs)** document why specific architectural approaches were selected.

Together, these artifacts provide complete traceability from business intent through architectural implementation decisions while preserving a clear separation of responsibilities.

Governance Going Forward

Future work should derive exclusively from this approved baseline.

Accordingly:

- Approved specification artifacts are treated as the authoritative source of system behavior.
- Architectural evolution should be documented through new ADRs whenever implementation decisions change without affecting approved behavior.
- Any change that affects externally observable behavior should first be reflected in the appropriate specification artifact before corresponding architectural documentation is updated.
- Traceability should be maintained across all future artifacts to preserve consistency with the approved baseline.

Project Milestone

This marks the completion of the project's Specification-Driven Development foundation. With the baseline established, subsequent work can proceed with a high degree of confidence that implementation, testing, and future evolution will remain anchored to a stable, reviewable, and versioned set of specifications.

It has been a pleasure working through this process with you. The resulting documentation set reflects a disciplined application of SDD principles, with strong traceability, clear separation between specification and implementation, and architectural governance that should scale well as the system evolves.